

Time Series Forecasting Project Report

Group ID: 88

Members: Hoang Nguyen Tu, Praneeth Mantina, Archil Jijavadze, Olha Melnyk

03/09/2026

Abstract

The project focuses on Time Series forecasting using a specialized architecture P-sLSTM (Parallel-Stacked Long Short-Term Memory). The forecasting task involved predicting measurements over a 96-step horizon, requiring the model to learn long-term temporal dependencies from the data provided. The dataset was preprocessed specifically for our task. Input sequences were structured as 336-step sliding windows, enabling the model to capture rich temporal context before generating predictions. Training was conducted with a number of epochs. The performance of model is measured using various loss functions such as Huber Loss (combination of Mean Squared Error (MSE) and Mean Absolute Error (MAE)). By leveraging the parallel-stacked LSTM structure, the model can effectively capture multi-scale temporal patterns and deliver a reliable long-horizon forecasts. As part of the requirement, we have used Jena climate Dataset as an additional dataset to measure our model's performance and also to know how well our trained model generalizes to the new dataset. PatchTST (on Original dataset) has performed significantly better compared to other Baseline models, achieving lowest MAE, MSE, RMSE, MAPE, SMAPE and WAPE. The model has demonstrated only around a 10% lower MAE in comparison to the same baselines on the new dataset.

1 Introduction

A multivariate time series forecasting is the estimation of future values of interdependent variables given data regarding their change over time. It is widely used in important fields such as economics and ecology through tasks such as predicting the stock market and specific environmental factors respectively. An example of the former is demonstrated in a study by Phuoc et al. (2024), which reports that through the use of long short-term memory (LSTM) Vietnamese stock market price trends have been predicted with an average accuracy of 93% across most of the stock data.

2 Related Work

Looking at both classical and modern methods, two architectures stood out. The first, transformers-based models, a more modern approach to time series forecasting (TSF), has seen great popularity in the field (Zerveas et al. 2021). It has solidified itself as one of the more prominent methods for

multivariate forecasting. A model proposed in the paper by Nie et al. (2023), called patch time series Transformer (PatchTST), is one of the most important papers published on this topic. The second, LSTM, is a type of recurrent neural network and a more traditional method that was one of the first used to tackle time series forecasting. While its performance has been overshadowed with newer architectures and models, its successor, extended LSTM (xLSTM) has seen promise in other fields (Beck et al. 2024). A paper by Kong et al. (2025) that modifies the said model to be specifically used for TSF datasets, has reported exceptional performance in comparison to other state-of-the-art architectures.

2.1 Patch Time Series Transformer

Patch time series transformer is a transformer model specifically designed for time series forecasting. At the time, its performance vastly overshadowed the state-of-the-art methods (Nie et al. 2023). Its main novelty, as stated in the name, is patching. Instead of defining a single timestep as a single token, it groups consecutive samples into patches. The advantages to this idea are various. By merging individual samples together into one token, this significantly reduces sequence length. In turn, this not only reduces the self-attention cost quadratically, but also lets the use of longer look-back windows to be manageable. Lastly, and most notably, through patching, the input tokens don't just describe individual samples, but the temporal variation in them. Although only local, these temporal patterns contain far more information than individual values. Another improvement over earlier models, which embedded all features together without separation, was weight sharing. PatchTST thinks of each feature as its own univariate sequence. It runs the transformer on each such sequence one by one and changes the weights in respect to the sum of their gradients. This way, it incorporates information from each feature individually, while keeping and learning only one set of weights. In contrast to other methods that changed the attention mechanism, for example the informer (Zhou et al. 2021) with prob-sparse attention to lower memory needs, PatchTST uses only classic attention. The logic behind it being that the vanilla implementation works applicably well if the input representation is defined correctly. The last major change in in terms of the model's pretraining. Taking a page out of Bert's method (Devlin et al. 2019), it uses masked prediction. However, due to patching, unlike masking individual samples, it now masks patches. After such pre-training, it can then be fine-tuned on specific forecasting datasets.

2.2 Parallel Stacked LSTM

Parallel Stacked Long Short-Term Memory is an enhanced LSTM-based architecture designed for **Time Series Forecasting**. It was proposed to address the limited feature extraction capability and information bottleneck of conventional sequential LSTM architectures, particularly when modeling complex non-linear and multivariate time series (Kong et al. 2025). Unlike traditional Recurrent Neural Networks (RNNs), which suffer from vanishing and exploding gradients when processing long sequences, LSTM employs memory cells and gating mechanisms to preserve long-term temporal dependencies (Hochreiter 1997). Building upon this foundation, P-sLSTM introduces multiple parallel stacked LSTM branches to learn temporal representations at different levels simultaneously, allowing the model to capture richer and more diverse temporal features while improving information flow across the network (Kong et al. 2025). This parallel architecture alleviates the information loss commonly observed in deep sequential LSTM models and improve forecasting ac-

curacy for complex datasets. containing non-linear and long-range dependencies. Consequently, P-sLSTM has demonstrated superior performance over traditional RNN architectures in various Time Series Forecasting tasks.

3 Method

The method of choice was to implement both above-mentioned architectures and train them with similar settings on the same dataset. The dataset used was the one provided in the lecture, albeit being preprocessed differently for the different models according to their respective guides. "Settings," in this case, mean the number of epochs and model parameters, most importantly the model size. For efficiency's sake, both of these were set to 30 and 128, respectively. The procedure then continued with only the better-performing model, the PatchTST.

3.1 Parameters & Training Procedure

Both models were trained using supervised window-based time-series forecasting. A context of 168 hourly observations was used to capture one week of historical information, with validation performed on a held-out period. Early stopping was used in both experiments, with the best model selected according to validation performance.

For P-sLSTM, the data were processed into overlapping patches of 24 time steps with a stride of 12. The model used 8 P-sLSTM blocks and produced a 336-step forecast. Training used MSE loss, a learning rate of 10^{-4} , batch size 32, and a maximum of 30 epochs. Early stopping was configured with a patience of 10 epochs.

For PatchTST, the same 168-step context and 24-step patches with stride 12 were used. The model was trained with Huber loss and the AdamW optimizer using a learning rate of 10^{-3} , weight decay of 10^{-5} , and batch size 256 for up to 30 epochs. Gradients were clipped to a norm of 1.0, and the learning rate followed a 200-step linear warm-up followed by cosine decay. Validation was performed using a 336-step rollout, with the checkpoint having the best validation RMSE retained. Early stopping used a patience of 10 epochs.

The two models therefore shared the same forecasting context and validation horizon but differed in architecture and optimization strategy. P-sLSTM used MSE optimization, while PatchTST used Huber loss together with AdamW, weight decay, gradient clipping, and a learning-rate schedule.

For further details on the parameters of both models, see Tables 1 and 2.

Table 1: P-sLSTM training and model parameters.

Parameter	Value
Input sequence length	168
Label length	24
Prediction length	336
Number of channels	23
Embedding dimension	128
Number of P-sLSTM heads	2
Conv1D kernel size	8
Group normalization weights	True
Dropout	0.1
Number of P-sLSTM blocks	8
Patch size	24
Patch stride	12
LSTM layers	1
Model dimension (d_{model})	512
Attention heads	8
Encoder layers	2
Decoder layers	1
Feed-forward dimension	2048
Moving average window	25
Attention factor	1
Encoder distillation	True
Temporal embedding	timeF
Activation	GELU
Batch size	32
Training epochs	30
Early stopping patience	10
Learning rate	1×10^{-4}
Loss function	MSE
Number of data-loader workers	4
Automatic mixed precision	False
GPU	CUDA:0
Random seed	2021

Table 2: PatchTST training and model parameters.

Parameter	Value
Context length	168
Patch length	24
Patch stride	12
Model dimension (d_{model})	128
Number of attention heads	8
Number of Transformer layers	8
Feed-forward dimension	64
Series embedding dimension	4
Dropout	0.1
Batch size	256
Training epochs	30
Learning rate	1×10^{-3}
Weight decay	1×10^{-5}
Loss function	Huber loss
Optimizer	AdamW
Warm-up steps	200
Learning-rate schedule	Linear warm-up + cosine decay
Minimum learning-rate ratio	0
Gradient clipping norm	1.0
Validation rollout	336
Early stopping patience	10
Random seed	42
Device	CUDA (if available)

3.2 Additional Dataset

The Dataset Jena Climate (2004-2016) was used for further validation. It is a multivariate weather time-series collected at Max Planck Institute for Biogeochemistry in Jena, Germany. The dataset is commonly used Predict future sequences from past sequences and also detect unusual weather patterns. The dataset is highly suitable for deep learning forecasting because of its Long, Continuous time series. The data has been recorded every 10 minutes from 2009 to 2016 (Max Planck Institute for Biogeochemistry 2017).

All Data preprocessing steps mentioned below are implemented in order to modify the dataset and make sure our model gets benefited from changes. We have created a new "df_hourly", an hourly-averaged version of the original 10-minute data, which reduces noise and highlights broader atmospheric patterns. After this, the model gains temporal awareness through "time features" such as hour, day, and month, and through "cyclic features" that encode repeating daily and yearly cycles using sine and cosine so the model understands that time wraps around. To capture temporal dependencies, lag features provide past values like temperature one hour or one day earlier, while "rolling features" summarize short-term trends and volatility using moving averages or moving standard deviations.

Once these features are added, "scaling" normalizes all variables to comparable ranges, ensuring

stable and efficient neural-network training. The dataset is then converted into model-ready sequences through windowing, which slices the continuous time series into fixed-length input windows paired with future targets. Finally, Dataset will be divided into training and test datasets, in order to check the model’s performance on unseen data.

3.3 Pre-processing for P-sLSTM and the Original Dataset

The P-sLSTM model was trained using the provided dataset with a series of pre-processing steps. First, missing values were handled through linear interpolation to preserve the temporal continuity of the time series. Next, the numerical features were standardized using Standard Deviation to achieve zero mean and unit variance, facilitating stable and efficient model training. Finally, the categorical identifier representing the series ID was transformed into numerical labels using label encoding, enabling the model to distinguish between different time series while maintaining computational efficiency.

3.4 Pre-processing for Patchtst and the Original Dataset

Before training with PatchTST, We preprocessed time-series data to ensure consistent input scales and to incorporate temporal information. First, the timestamp variable was converted into several cyclical time features, including sine and cosine representations of the hour of day, day of week, and day of year. This allows the model to capture periodic temporal patterns while preserving the cyclic relationship between adjacent time points. The numerical input features were then standardized using their mean and standard deviation computed from the training data. Missing values in the input features were replaced by the corresponding training-set mean before standardization.

The target variable was normalized separately for each different series using the corresponding series-specific mean and standard deviation. This reduces scale differences between individual time series. For previously unseen series, global target statistics were used as a fallback. The series’ id was converted into an integer index and used as a categorical identifier. Finally, the data were sorted by series and timestamp, and transformed into autoregressive sliding windows. For a given context length, the model receives the normalized target history together with the normalized feature values from the preceding time steps and predicts the target at the next time step. The preprocessing statistics were stored as part of the preprocessing state so that the same transformations could be consistently applied during validation and inference.

4 Experiments

4.1 PatchTST vs. P-sLSTM

Evaluation of performance of smaller models of PatchTST and P-sLSTM has demonstrated that the former wins by a decent margin irrelevant of the loss type used (See Figures 1 and 2). They were both tested with the same parameters, the most important, `d_models` and `epoch count`, being 128 and 30 respectively. We assume that the newer model, the P-sLSTM, has not performed on the

level of PatchTST due to overfitting from the start. To address this issue, we have reinitiated this experiment again with optimized parameters for both of the models. For PatchTST the optimal changes were reducing Transformer depth from 8 to 4 layers and increasing d_{ff} from 64 to 256. For P-sLSTM, we gave it a much longer context of 672 hours, used only 1 block rather than 8, changed the patch structure to 56/56 non-overlapping patches and added more regularization with 0.2 dropout and 10^{-4} weight decay. In terms of PatchTST, the validation performance has increased by a decent margin (See Fig. 3). However, while the validation loss has decreased for P-sLSTM, it still demonstrated major signs of overfitting (See Fig. 4).

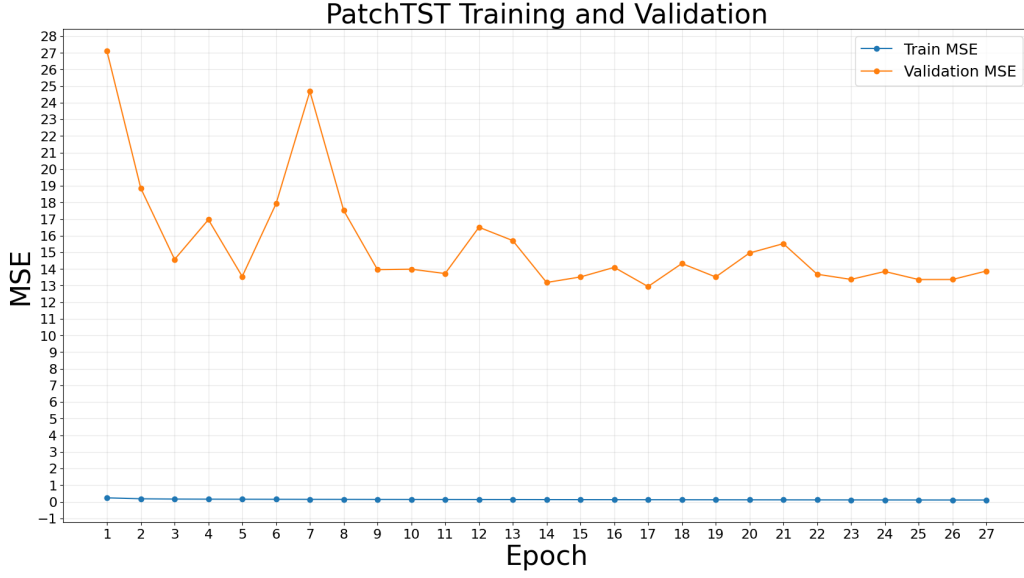


Figure 1: First PatchTST model training and validation metrics.

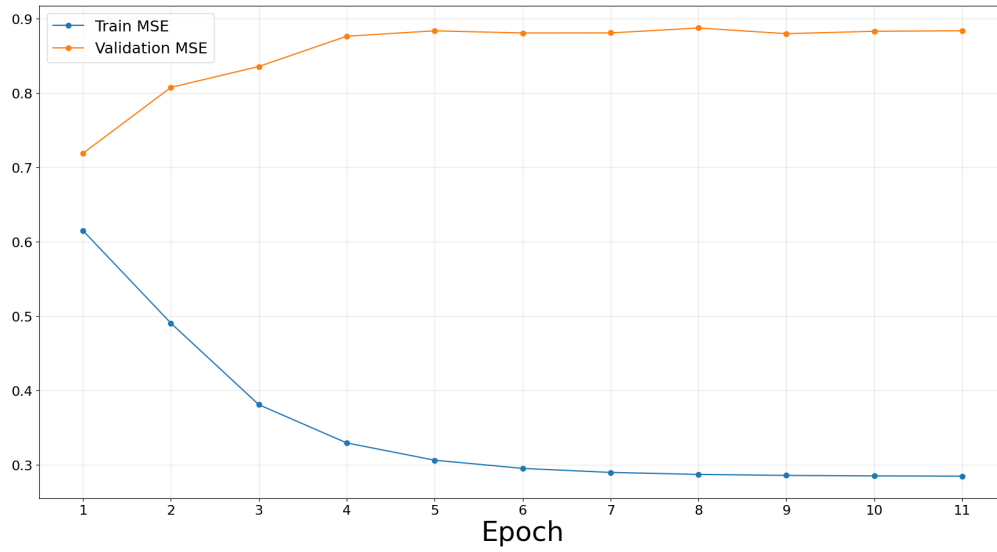


Figure 2: First P-sLSTM model training and validation metrics. The training has stopped after 11 epochs due to patience.

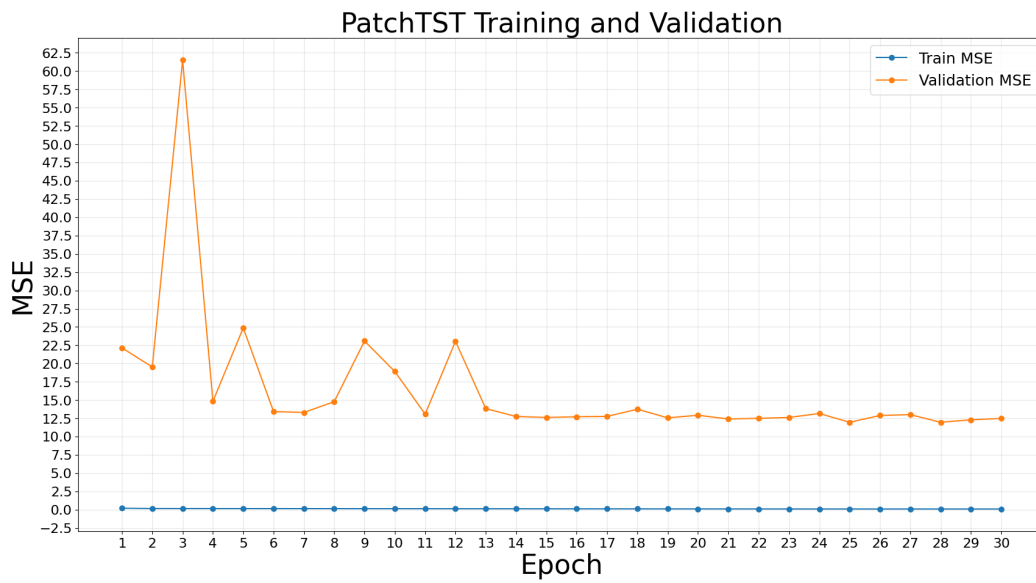


Figure 3: Best PatchTST training and validation metrics.

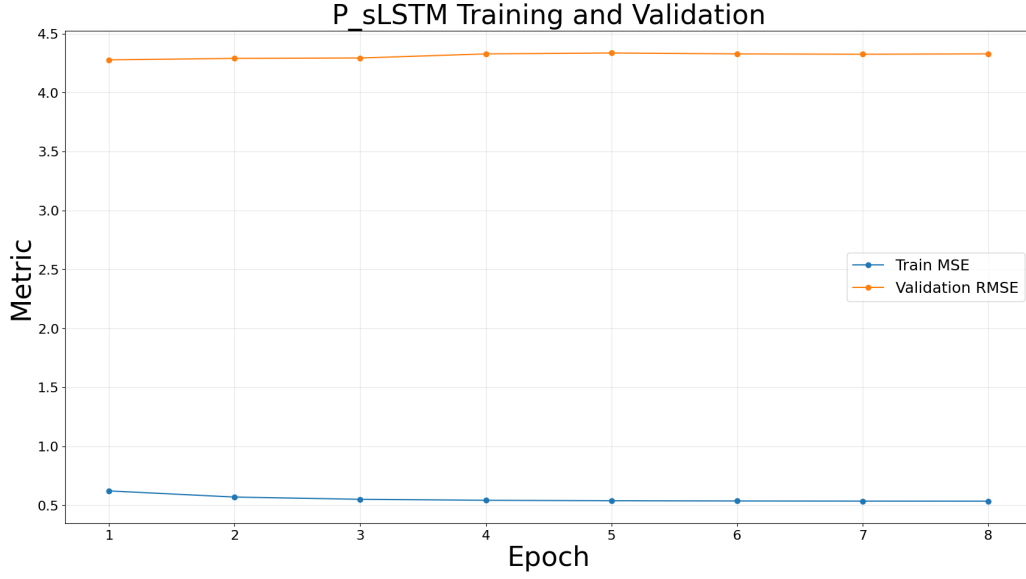


Figure 4: Best P-sLSTM training and validation metrics. The training has stopped after 8 epochs due to patience.

4.2 PatchTST Final Model Results

The model used for the final results is the best-performing PatchTST model from the previous experiment. It significantly outperformed all baseline models on the original dataset; further details are presented in Figure 5. Although its performance was less pronounced on the new dataset, it still achieved approximately 10% lower MAE than the baseline models (see Figure 7).

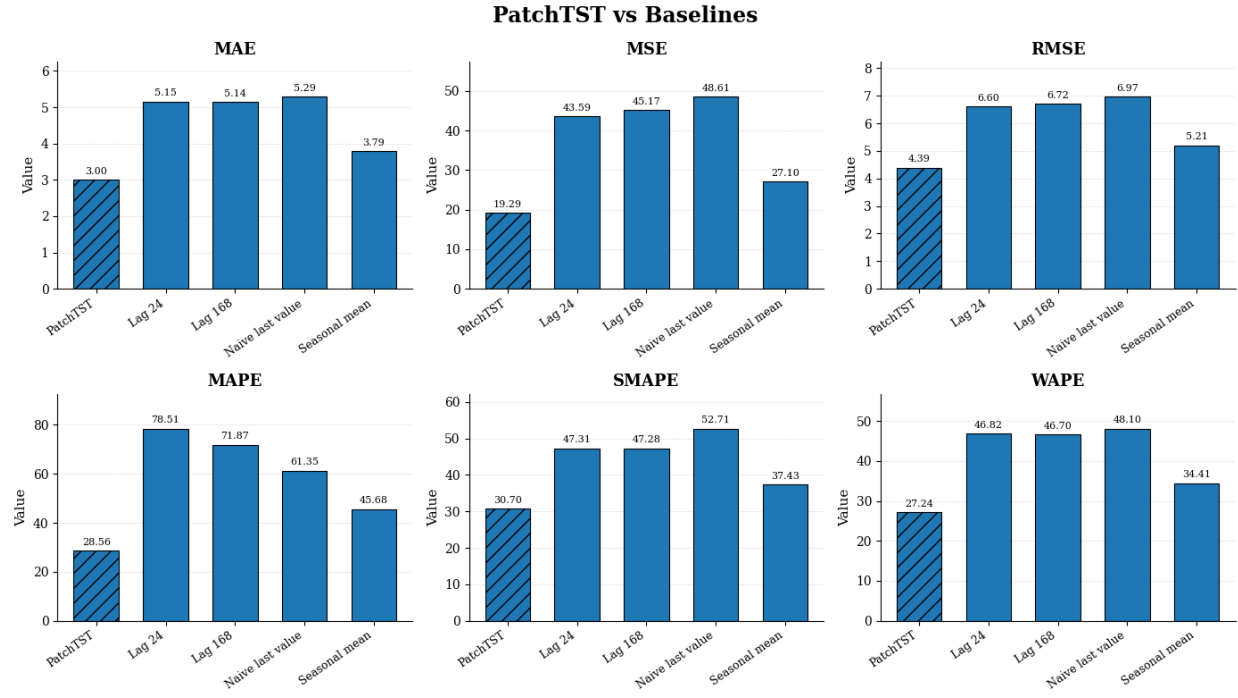


Figure 5: Baselines vs. PatchTST on the original dataset.

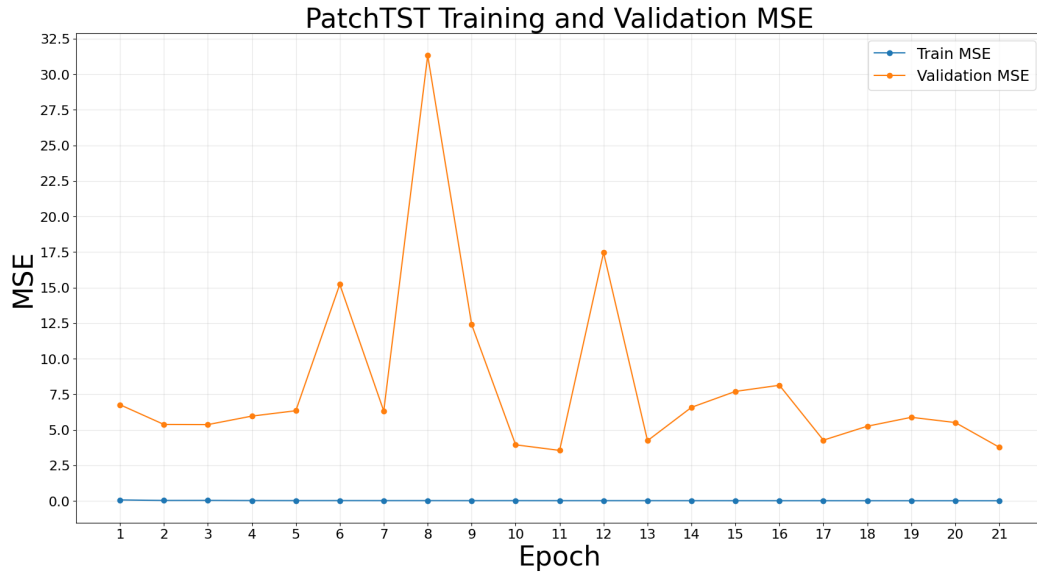


Figure 6: PatchTST model training and validation on the new dataset.

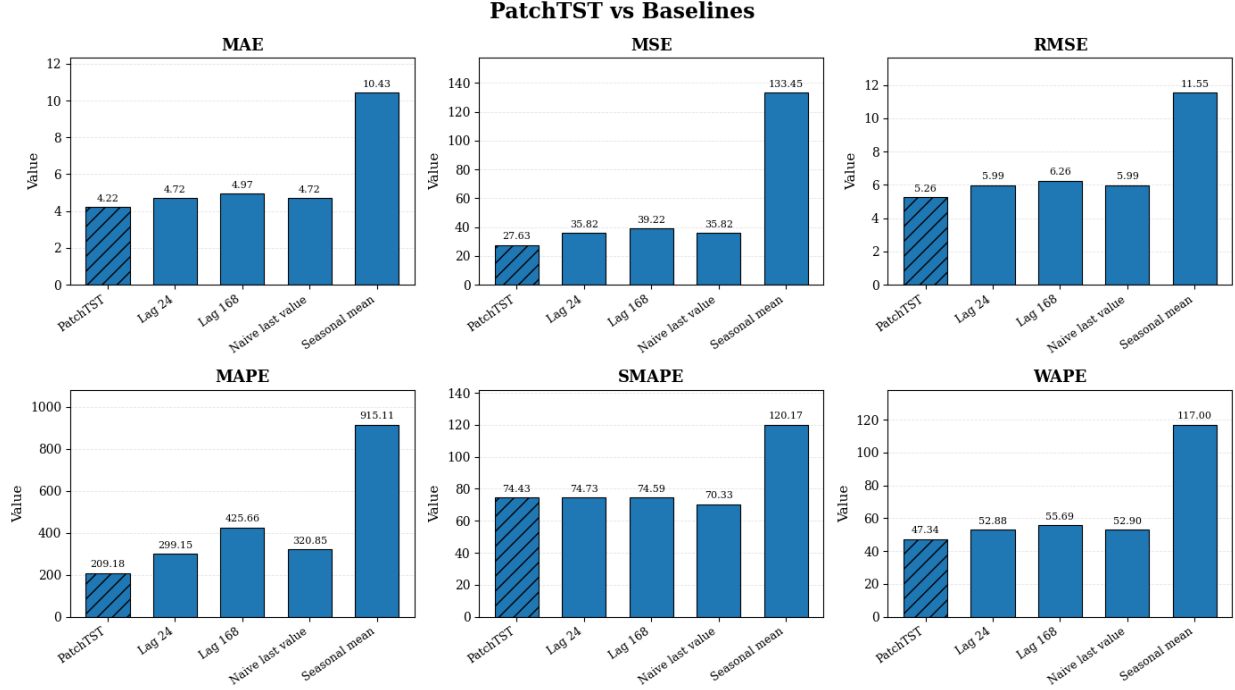


Figure 7: Baselines vs. PatchTST on the new dataset.

5 Conclusion

The evaluation showed that PatchTST consistently outperformed P-sLSTM, regardless of the loss function used. Both models were initially tested with the same settings, but P-sLSTM appeared to suffer from significant overfitting. A second experiment with adjusted parameters improved PatchTST’s validation performance and reduced P-sLSTM’s validation loss, although P-sLSTM continued to show substantial overfitting.

For the final results, the best-performing PatchTST model was selected. It significantly outperformed all baseline models on the original dataset and also achieved superior performance on a separate dataset, suggesting that PatchTST has strong generalization capabilities.

Contributions

Hoang Nguyen Tu: preprocessed provided dataset for P-sLSTM, studied and implemented P-sLSTM, conducted training and validation graphing for P-sLSTM and PatchTST.

Praneeth Mantina: Research and Selection of additional dataset (Jena climate dataset) for validating our model, Contributed in Data preprocessing and Evaluation of Jena climate dataset, Chapter 3.2 and Abstract.

A. Jijavadze: Contributed in writing chapters 1, 2, 3, 4, 5. PatchTST and P-sLSTM train-

ing/validation graphing. Final PatchTST vs Baselines graphing. Creating communication medium, scheduling meetings, directing development. Contributions in parameter tuning for training.

Olha Melnyk: Trained and validated the PatchTST and P-sLSTM models, researched and experimented with hyperparameter tuning, and contributed to preprocessing of both the initial dataset and the Jena climate dataset.

References

- Beck, Maximilian et al. (2024). “xlstm: Extended long short-term memory”. In: *Advances in Neural Information Processing Systems* 37, pp. 107547–107603.
- Devlin, Jacob et al. (2019). “Bert: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186.
- Hochreiter, S (1997). “Long short-term memory”. In: *Neural Computation MIT-Press*.
- Kong, Yaxuan et al. (2025). “Unlocking the power of lstm for long term time series forecasting”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 39. 11, pp. 11968–11976.
- Max Planck Institute for Biogeochemistry (2017). *Jena Climate 2009–2016: Multivariate Weather Time Series*. MPI-BGC Weather Station, Jena, Germany.
- Nie, Yuqi et al. (2023). “A time series is worth 64 words: Long-term forecasting with transformers”. In: *International Conference on Learning Representations*.
- Phuoc, Tran et al. (2024). “Applying machine learning algorithms to predict the stock price trend in the stock market—The case of Vietnam”. In: *Humanities and Social Sciences Communications* 11.1, pp. 1–18.
- Zerveas, George et al. (2021). “A transformer-based framework for multivariate time series representation learning”. In: *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pp. 2114–2124.
- Zhou, Haoyi et al. (2021). “Informer: Beyond efficient transformer for long sequence time-series forecasting”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 35. 12, pp. 11106–11115.